MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of Faculty: Dr. Neeraj Jain
Subject: Computer Organization & Architecture (5AD4-02)
Semester: V **Session: 2023-24 (ODD Sem.)**
Branch: AI & DS **Batch: A**
Unit: 1 **Date of Submission: 19/10/2023**

Lecture No.	Topics Covered	Page No.	Total Page
1	Objective , Scope & Outcome	1-4	4
1	Register Transfer Language (RTL)	5-8	4
2	Bus Transfer	9-10	2
3	Memory Transfer	11-15	5
5	Arithmetic Micro-Operation	16-18	3
6	Logic Micro -operation	19-21	3
7	Shift –Micro-operation	22-23	2
8	Arithmetic Logic Shift Unit (ALU)	24-25	2

Total Pages: 25

References

1. M. Morris Mano, Computer System Architecture, Pearson
2. William Stallings, Computer Organization & Architecture, Pearson
3. Tannenbum, Structured Computer Organization, PHI

19/10/2023

Faculty **HOD** **Dean Academics**

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Objective, Scope and Outcome

LECTURE NO:-1

Computer Architecture

Computer Architecture deals with giving operational attributes of the computer or Processor to be specific. It deals with details like physical memory, ISA (Instruction Set Architecture) of the processor, the number of bits used to represent the data types, Input Output mechanism and technique for addressing memories.

Computer Organization

Computer Organization is realization of what is specified by the computer architecture .It deals with how operational attributes are linked together to meet the requirements specified by computer architecture. Some organizational attributes are hardware details, control signals, peripherals.

EXAMPLE

Say you are in a company that manufactures cars, design and all low-level details of the car come under computer architecture (abstract, programmers view), while making it's parts piece by piece and connecting together the different components of that car by keeping the basic design in mind comes under computer organization (physical and visible).

Computer Organization	Computer Architecture
Transparent from programmer (ex. a programmer does not worry much how addition is implemented in hardware)	Programmer view (i.e. Programmer has to be aware of which instruction set used)

Physical components (Circuit design, Adders, Signals, Peripherals)	Logic (Instruction set, Addressing modes, Data types, Cache optimization)
How to do? (implementation of the architecture)	What to do ? (Instruction set)

GENERATIONS OF A COMPUTER

Generation in computer terminology is a change in technology a computer was being used. Initially, the generation term was used to distinguish between varying hardware technologies. But nowadays, generation includes both hardware and software, which together make up an entire computer system.

There are totally five computer generations known till date. Here approximate dates against each generations have been mentioned which are normally accepted.

Following are the main five generations of computers:

S.N.	Generation & Description
1	**First Generation** The period of first generation: 1946-1959. Vacuum tube based.
2	**Second Generation** The period of second generation: 1959-1965. Transistor based.
3	**Third Generation** The period of third generation: 1965-1971. Integrated Circuit based.
4	**Fourth Generation** The period of fourth generation: 1971-1980. VLSI microprocessor based.
5	**Fifth Generation** The period of fifth generation: 1980-onwards. ULSI microprocessor based

Classification digital Computer based on size and Capability

Based on size and capability, computers are broadly classified into:

[1] Micro Computers (Personal Computer)

A microcomputer is the smallest general purpose processing system. The older pc started 8 bit processor with speed of 3.7MB and current pc 64 bit processor with speed of 4.66 GB.

Examples: - **IBM PC**s, **APPLE** computers

Microcomputer can be classified into 2 types:

1. Desktops

2. Portables

The difference is portables can be used while travelling whereas desktops computers cannot be carried around.

[2] Minicomputer

A minicomputer is a medium-sized computer. That is more powerful than a microcomputer. These computers are usually designed to serve multiple users simultaneously (Parallel Processing). They are more expensive than microcomputers.

Examples: Digital Alpha, Sun Ultra.



Fig. 1.1 Minicomputer

[3] Mainframe (Enterprise) computers

Computers with large storage capacities and very high speed of processing (compared to mini- or microcomputers) are known as mainframe computers. They support a large number of terminals for simultaneous use by a number of users like ATM transactions. They are also used as central host computers in distributed data processing system.

Examples: - **IBM 370, S/390.**



Fig. 1.2 Mainframe computers

References

1. Carl Hamacher, Zvonks Vranesic, SafeaZaky (2002), Computer Organization, 5th edition, McGraw Hill

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Register Transfer Language (RTL)

LECTURE NO:-2

The term "Register Transfer" can perform micro-operations *and transfer the result of operation to the same or other register.*

Micro-operations

The operation performed on the data store in registers are called micro-operations. They are low-level instructions used in designs of complex machine instructions.

Register Transfer

The information transformed from one register to another register is represented in symbolic form by replacement operator is called Register Transfer.

Replacement Operator

In the statement, R2 <- R1, acts as a replacement operator. This statement defines the transfer of content of register R1 into register R2.

There are various methods of RTL as shown in Fig. 1.3.

Fig (a) General way of representing a register is by the name of the register enclosed in a rectangular box

Fig. (b) Register is numbered in a sequence of 0 &1.

Fig. © The numbering of bits in a register can be marked on the top of the box

Fig. (d) A 16-bit register PC is divided into 2 parts- Bits (0 to 7) are assigned with lower byte of 16-bit address and bits (8 to 15) are assigned with higher bytes of 16-bit address.

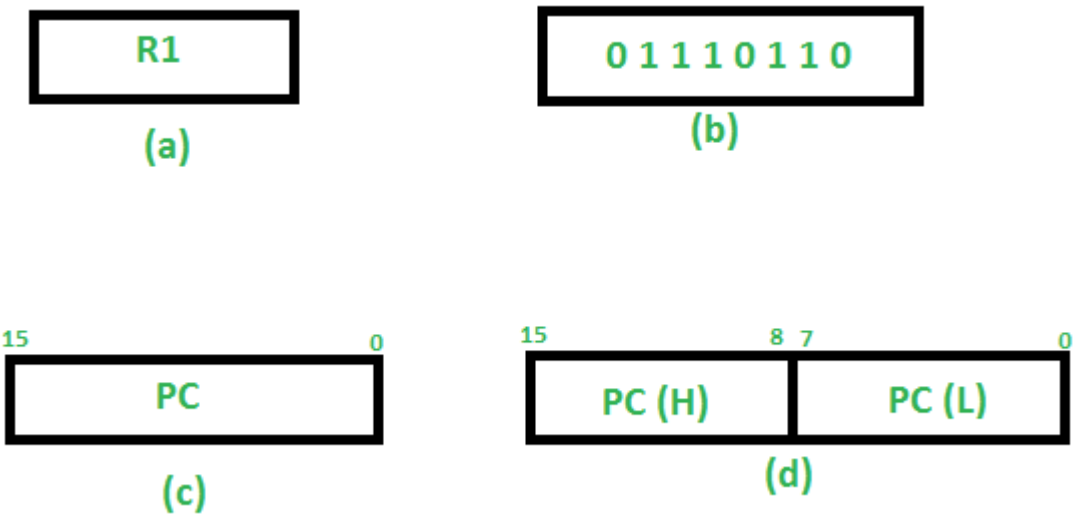


Fig. 1.3 Register Transfer Methods

Register Transfer Operations:

The operation performed on the data stored in the registers are referred to as register transfer operations.

There are different types of register transfer operations:

1. Simple Transfer (R2 <- R1)

The content of R1 are copied into R2 without affecting the content of R1. It is an unconditional type of transfer operation.

2. Conditional Transfer

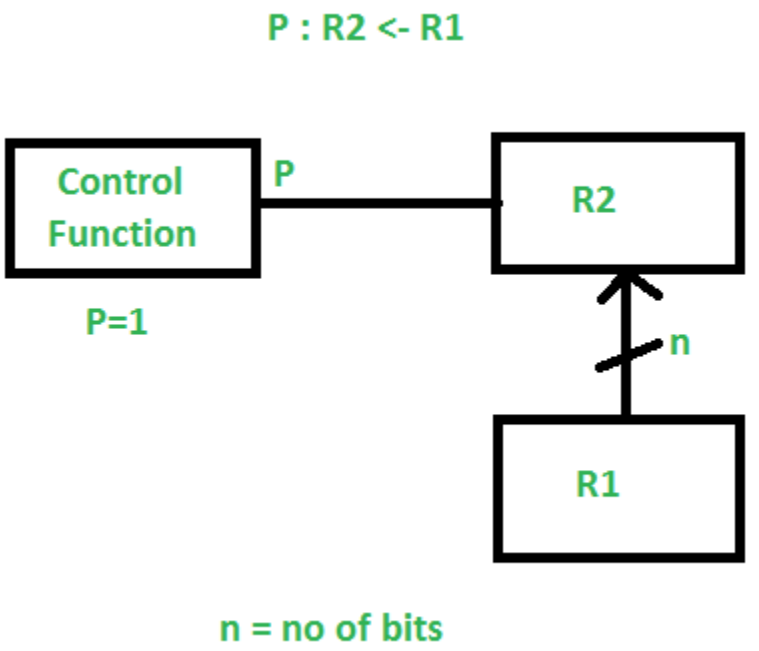


Fig. 1.4 Conditional Transfer Register

It indicates that if P=1, then the content of R1 is transferred to R2. It is a unidirectional operation.

3. Simultaneous Operations

If 2 or more operations are to occur simultaneously then they are separated with comma **(,)**.

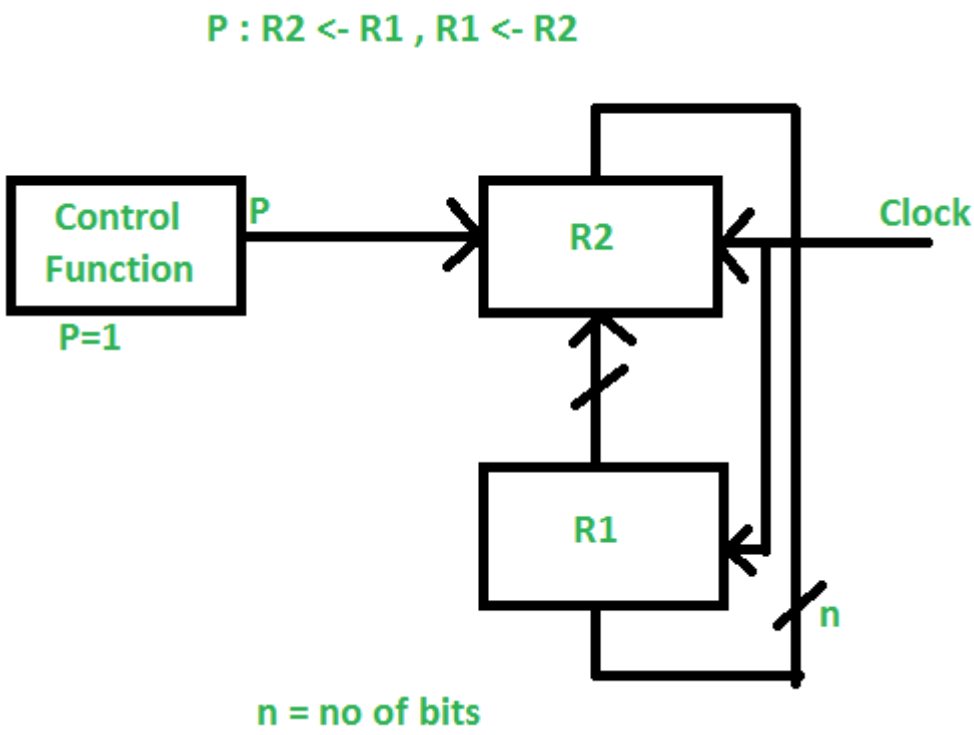


Fig. 1.5 Simultaneous Transfer Register

References:

1. https://www.geeksforgeeks.org/register-transfer-language-rtl/

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Bus Transfer

LECTURE NO:-3

BUS STRUCTURES

Group of lines that serve as connecting path for several devices is called a bus (one bit per line).

A bus is basically a subsystem which transfers data between the components of Computer components either within a computer or between two computers. It connects peripheral devices at the same time.

- A multiple Bus Structure has multiple inter connected service integration buses and for each bus the other buses are its foreign buses. A Single bus structure is very simple and consists of a single server.

- A bus cannot span multiple cells. And each cell can have more than one buses. - Published messages are printed on it. There is no messaging engine on Single bus structure

I) In single bus structure all units are connected in the same bus while connecting different buses as multiple bus structure.

II) Multiple bus structure's performance is better than single bus structure.

Iii) Single bus structure's cost is cheap than multiple bus structure.

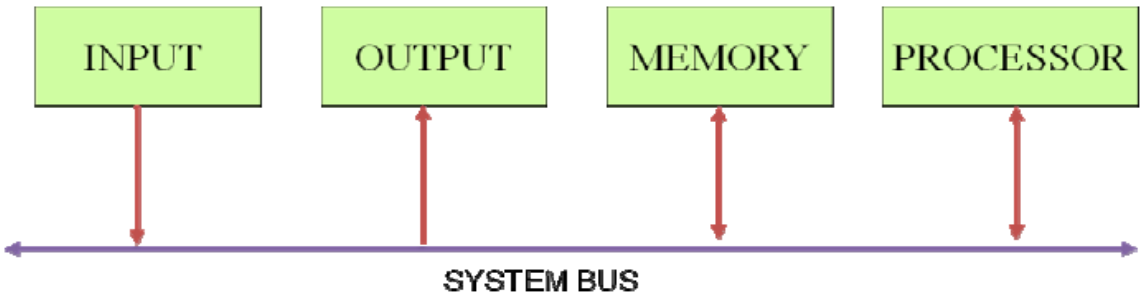


Figure 5: Single bus structure

Types of Buses

1. Data Bus

Data bus is the most common type of bus. It is used to transfer data between different components of computer. The number of lines in data bus affects the speed of data transfer between different components. The data bus consists of 8, 16, 32, or 64 lines. A 64-line data bus can transfer 64 bits of data at one time.

The data bus lines are bi-directional. It means that CPU can read data from memory using these lines CPU can write data to memory locations using these lines

2. Address Bus

Many components are connected to one another through buses. Each component is assigned a unique ID. This ID is called the address of that component. It a component wants to communicate with another component, it uses address bus to specify the address of that component. The address bus is a unidirectional bus. It can carry information only in one direction. It carries address of memory location from microprocessor to the main memory.

3. Control Bus

Control bus is used to transmit different commands or control signals from one component to another component. Suppose CPU wants to read data from main memory. It will use control is also used to transmit control signals like ASKS (Acknowledgement signals).

References:

1. https://www.geeksforgeeks.org/register-transfer-language-rtl/

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Memory Transfer

LECTURE NO:-4

-A digital computer has many registers, and paths must be provided to transfer information from one register to another.

-The number of wires will be excessive if separate lines are used between each register.

-A more efficient scheme for transferring information between registers in a multiple-register configuration is a common bus system.

-A bus structure consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time.

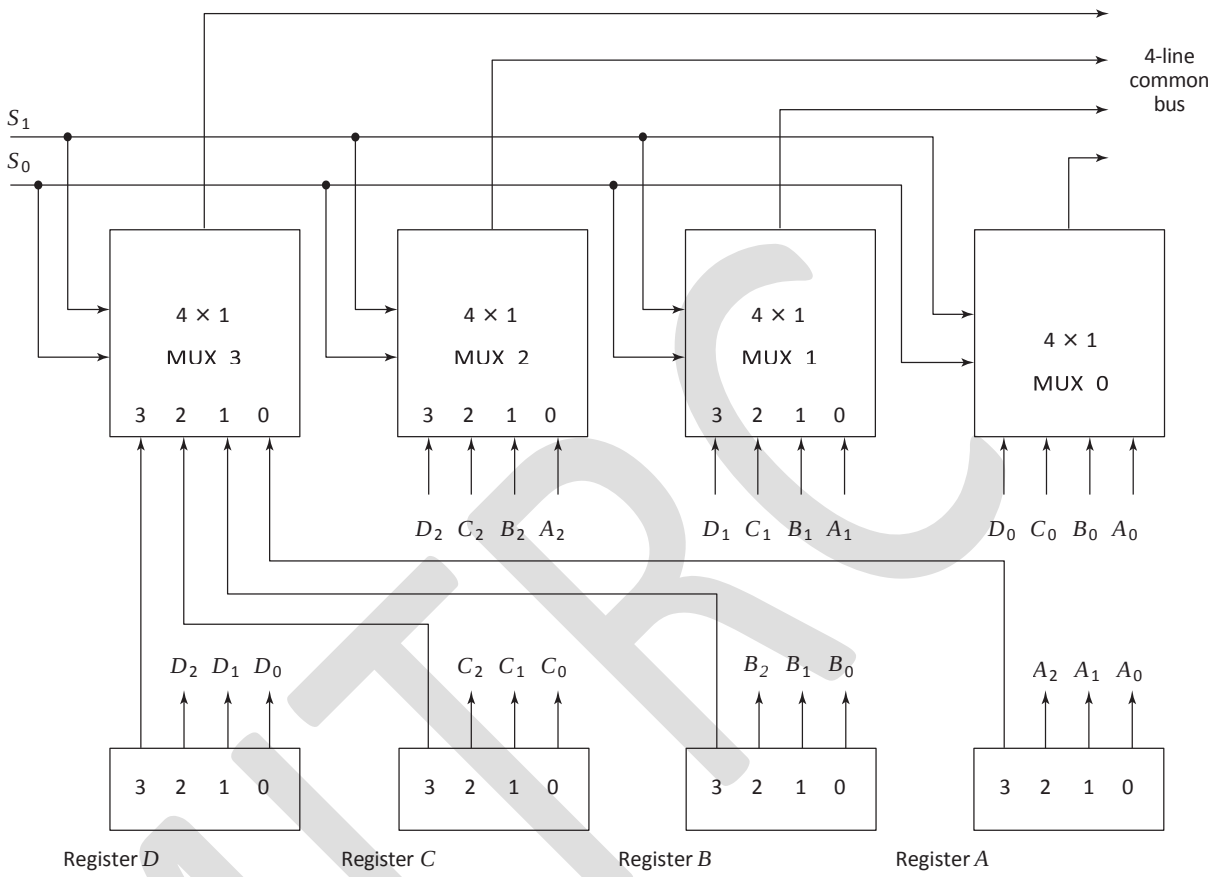


Fig. 1.7 Bus system for four registers

-Control signals determine which register is selected by the bus during each particular register transfer.

-One way of constructing a common bus system is with multiplexers. The multiplexers select the source register whose binary information is then placed on the bus.

The two selection lines S_1 and S_0 are connected to the selection inputs of all four multiplexers. The selection lines choose the four bits of one register and transfer them into the four line common bus.

When S_1S_0 = 00, the 0 data inputs of all four multiplexers are selected and applied to the outputs that form the bus. This causes the bus lines to receive the content of register *A* since the outputs of this register are connected to the 0 data inputs of the multiplexers. Similarly, register *B* is selected if S_1S_0 = 01, and so on.

Table 1.1 shows the register that is selected by the bus for each of the four possible binary value of the selection lines.

Table 1.1 Function Table for Bus of Fig. 1.7

S_1	S_0	Register selected
0	0	*A*
0	1	*B*
1	0	*C*
1	1	*D*

The transfer of information from a bus into one of many destination registers can be accomplished by connecting the bus lines to the inputs of all destination registers and activating the load control of the particular destination register selected.

The symbolic statement for a bus transfer may mention the bus or its presence may be implied in the statement.

$$BUS \leftarrow C, \quad R1 \leftarrow BUS$$

The content of register *C* is placed on the bus, and the content of the bus is loaded into register *R*1 by activating its load control input.

Three-State Bus Buffers

A bus system can be constructed with three-state gates instead of multiplex- ers. A three-state gate is a digital circuit that exhibits three states. Two of the states are signals equivalent to logic 1 and 0 as in a conventional gate. The third state is a *high-impedance state*. The high-impedance state behaves like an open circuit, which means that the output is disconnected and does not have a logic significance.

The graphic symbol of a three-state buffer gate is shown in Fig. 1.8. It is distinguished from a normal buffer by having both a normal input and a control input. The control input determines the output state. When the control input is equal to 1, the output is enabled and the gate behaves like any conventional buffer, with the output equal to the normal input. When the control input is 0, the output is disabled and the gate goes to a high-impedance state, regardless of the value in the normal input. The high-impedance state of a three-state gate provides a special feature not available in other gates.

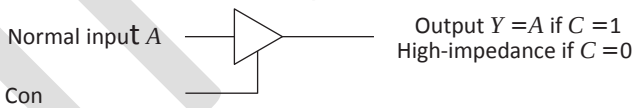
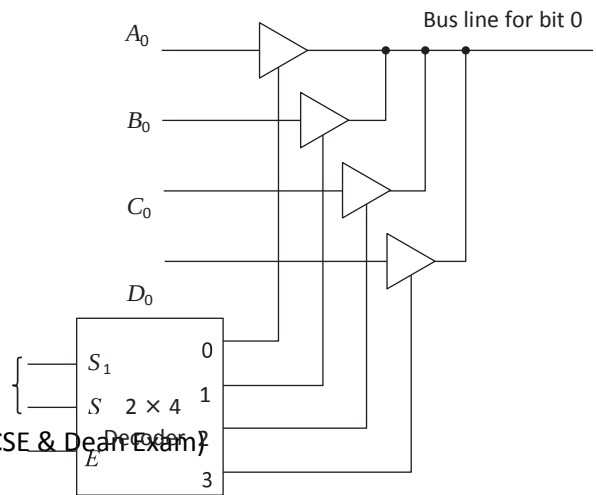


Fig. 1.8 Graphic symbols for three-state buffer.



Enable

Fig. 1.9 Bus line with three state-buffers

As shown in Fig. 1.9 only one decoder is necessary to select between the four registers.

Memory Transfer

The transfer of information from a memory word to the outside environment is called a *read* operation. The transfer of new information to be stored into the memory is called a *write* operation. A memory word will be symbolized by the letter *M*.

Consider a memory unit that receives the address from a register, called the address register, symbolized by *AR* . The data are transferred to another register, called the data register, symbolized by *DR*. The read operation can be stated as follows:

Read: $DR \leftarrow M[AR]$

This causes a transfer of information into DR from the memory word M selected by the address in AR.

The write operation can be stated symbolically as follows:

Write: $M[AR] \leftarrow R1$

This causes a transfer of information from *R*1 into the memory word *M* selected by the address in *AR*.

References:

1. M. Morris Mano, Computer System Architecture, Pearson

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Arithmetic Micro- Operations

LECTURE NO:-5

The basic arithmetic micro operations are addition, subtraction, increment, decrement, and shift.

add microoperation

$$R3 \leftarrow R1 + R2$$

It states that the contents of register R 1 are added to the contents of register R 2 and the sum transferred to register $R3$. To implement this statement with hardware we need three registers and the digital component that performs the addition operation.

Subtraction microoperation

It is most often implemented through complementation and addition. Instead of using the minus operator, we can specify the subtraction by the following statement:

$$R3 \leftarrow R1 + \overline{R2} + 1$$

$\overline{R2}$ is the symbol for the 1's complement of R 2. Adding 1 to the 1's comple- ment produces the 2's complement. Adding the contents of $R1$ to the 2's com- plement of R 2 is equivalent to $R1$ — $R2$.

The increment and decrement microoperations

$R1 \leftarrow R1 + 1$	Increment the contents of $R1$ by one
$R1 \leftarrow R1 - 1$	Decrement the contents of $R1$ by one

Binary Adder

To implement the add microoperation with hardware, we need the registers that hold the data and the digital component that performs the arithmetic addition. The digital circuit that forms the arithmetic sum of two bits and a previous carry is called a full-adder

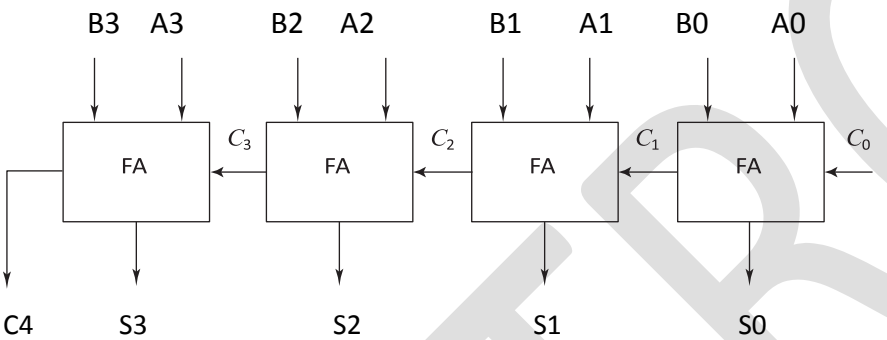


Fig. 1.10 4-bit binary adder

The binary adder is constructed with full-adder circuits connected in cascade, with the output carry from one full-adder connected to the input carry of the next full-adder.

Binary Adder-Subtractor

The addition and subtraction operations can be combined into one com- mon circuit by including an exclusive-OR gate with each full-adder. A 4-bit adder-subtractor circuit is shown in Fig. 1.11. The mode input M controls the operation. When $M = 0$ the circuit is an adder and when $M = 1$ the circuit becomes a subtractor.

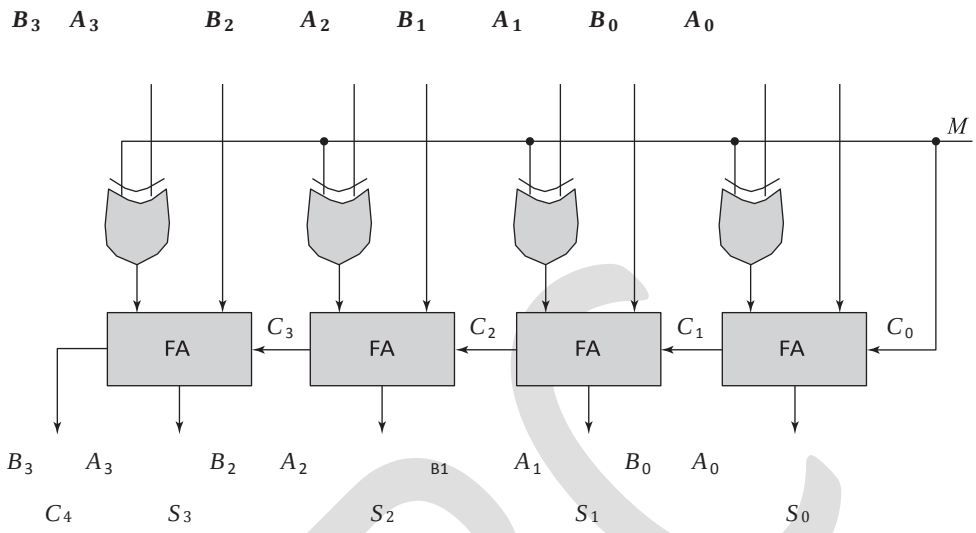


Fig. 1.11 4-bit adder-subtractor.

Binary Incrementer

The increment microoperation adds one to a number in a register. For exam- ple, if a 4-bit register has a binary value 0110, it will go to 0111 after it is incremented.

The diagram of a 4-bit combinational circuit incrementer is shown in Fig. 1.12.

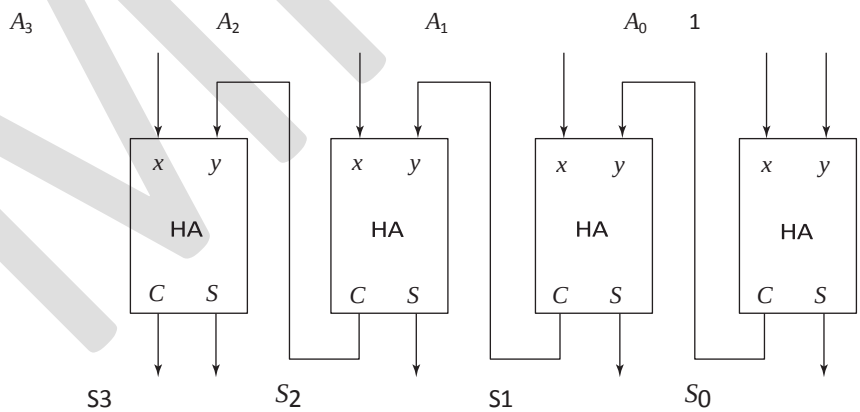


Fig. 1.12 4-bit binary incrementer

Reference:

1. M. Morris Mano, Computer System Architecture, Pearson

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Logic Micro- Operations

LECTURE NO:-6

Logic microoperations specify binary operations for strings of bits stored in registers. These operations consider each bit of the register separately and treat them as binary variables.

There are 16 different logic operations that can be performed with two binary variables. They can be determined from all possible truth tables obtained with two binary variables as shown in Table 1-2.

TABLE 1.2 Truth Tables for 16 Functions of Two Variables

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

In this table, each of the 16 columns F_0 through F_{15} represents a truth table of one possible Boolean function for the two variables x and y.

TABLE 1.3 Sixteen Logic Microoperations

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	

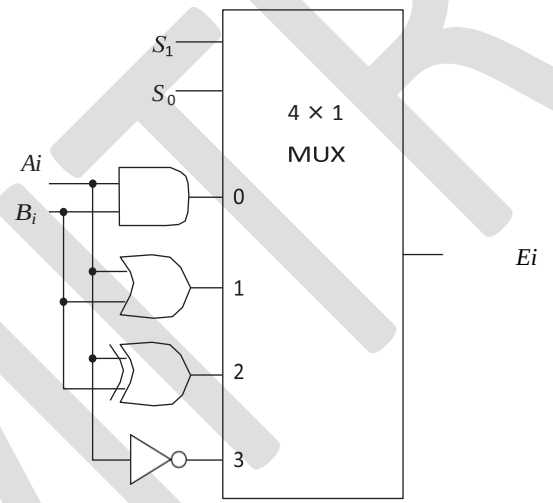
Remaining are given in Morris Mano Book.

Hardware Implementation

The hardware implementation of logic microoperations requires that logic gates be inserted for each bit or pair of bits in the registers to perform the required logic function. Although there are 16 logic microoperations, most computers use only four—AND, OR, XOR (exclusive-OR), and complement— from which all others can be derived.

Fig. 1.13 shows one stage of a circuit that generates the four basic logic microoperations. It consists of four gates and a multiplexer. Each of the four logic operations is generated through a gate that performs the required logic

Fig. 1.13 (a) One stage of logic circuit.



MITRC

Fig. 1.13 (b) Functional Table

S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \overline{A}$	Complement

Reference:

1. M. Morris Mano, Computer System Architecture, Pearson

MITRC

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Shift Micro- Operations

LECTURE NO:-7

Shift microoperations are used for serial transfer of data. There are three types of shifts: logical, circular, and arithmetic.

A *logical* shift is one that transfers 0 through the serial input. We will adopt, the symbols *shl* and *shr* for logical shift-left and shift-right microoperations. For example:

$$R1 \leftarrow \text{shl } R1$$

$$R2 \leftarrow \text{shr } R2$$

are two microoperations that specify a 1-bit shift to the left of the content of register *R*1 and a 1-bit shift to the right of the content of register *R*2.

The *circular* shift (also known as a *rotate* operation) circulates the bits of the register around the two ends without loss of information. This is accom- plished by connecting the serial output of the shift register to its serial input.

The symbolic notation for the shift microoperations is shown in Table 1-4.

TABLE 4-7 Shift Microoperations

Symbolic designation	Description
$R \leftarrow \text{shl } R$	Shift-left register R
$R \leftarrow \text{shr} R$	Shift-right register R
$R \leftarrow \text{cil } R$	Circular shift-left register R
$R \leftarrow \text{cir } R$	Circular shift-right register R
$R \leftarrow \text{ashl } R$	Arithmetic shift-left R
$R \leftarrow \text{ashr } R$	Arithmetic shift-right R

An arithmetic shift is a microoperation that shifts a signed binary number to the left or right. An arithmetic shift-left multiplies a signed binary number by 2. An arithmetic shift-right divides the number by 2. Arithmetic shifts must leave the sign bit unchanged because the sign of the number remains the same.

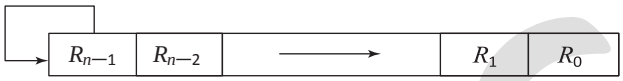


Fig. 1.14 Arithmetic shift right.

Reference:

1. M. Morris Mano, Computer System Architecture, Pearson

MITRC

UNIT-1 (REGISTER TRANSFER AND MICRO OPERATIONS)

Arithmetic Logic Shift Unit (ALU)

LECTURE NO:-8

Computers has two kinds of components:

Hardware, consisting of its physical devices (CPU, memory, bus, storage devices, ...)

Software, consisting of the programs it has (Operating system, applications, utilities, ...)

FUNCTIONAL UNIT

A computer consists of five functionally independent main parts input, memory, arithmetic logic unit (ALU), output and control unit.

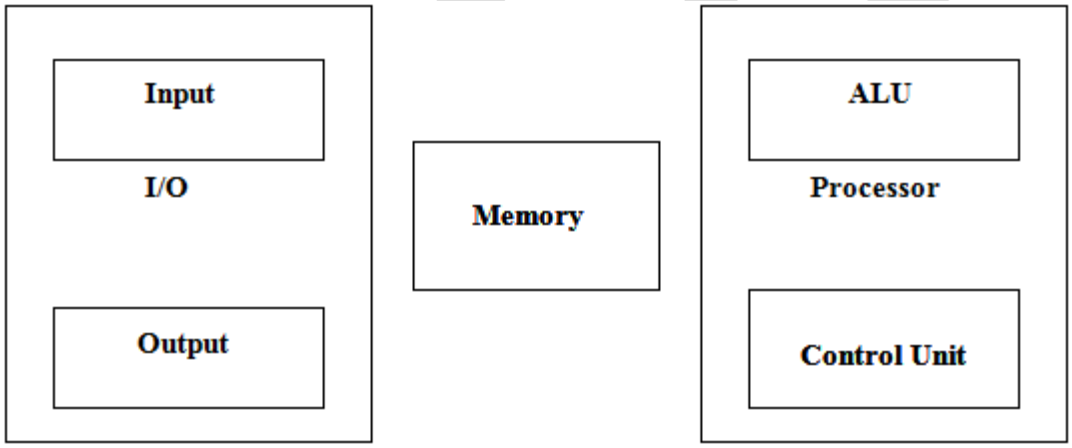
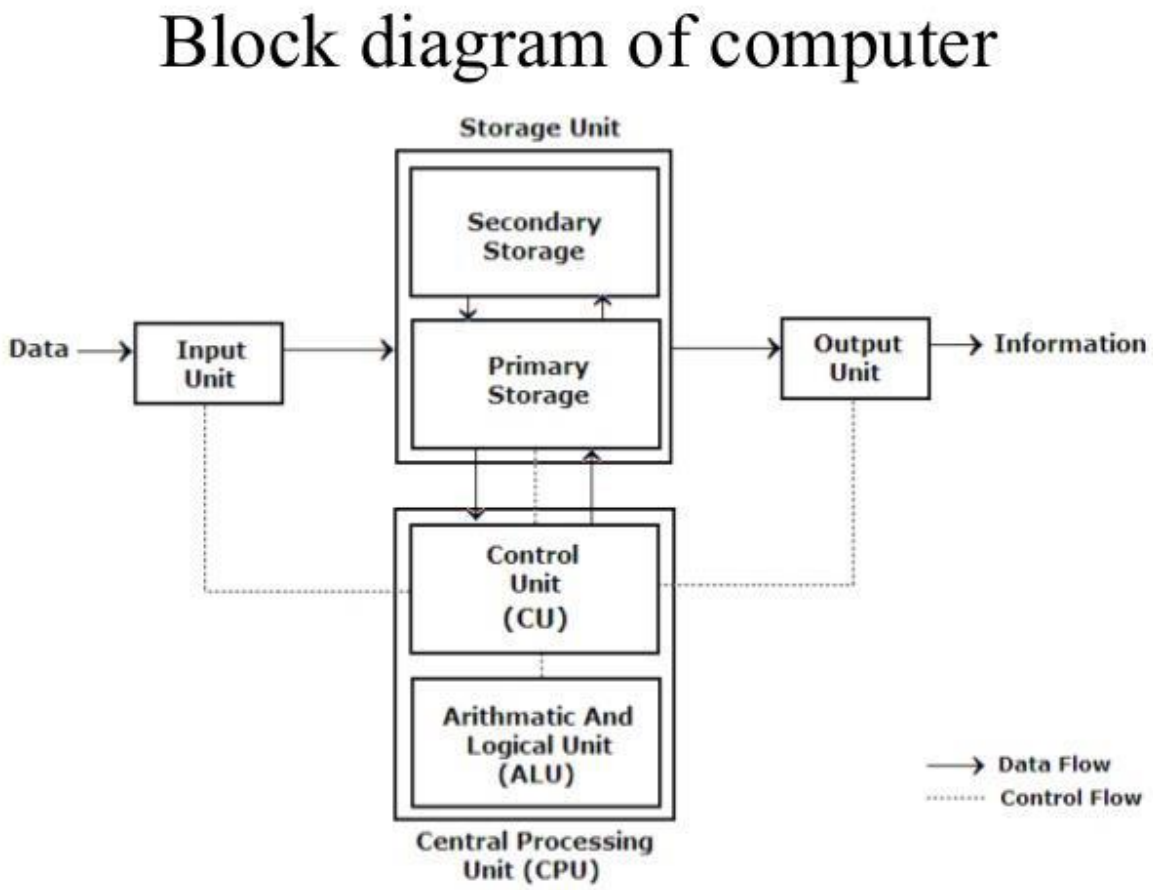


Fig. Functional units of computer

Input device accepts the coded information as source program i.e. high level language. This is either stored in the memory or immediately used by the processor to perform the desired operations. The program stored in the memory determines the processing steps. Basically the computer converts one source program to an object program. i.e. into machine language.

Finally the results are sent to the outside world through output device. All of these actions are coordinated by the control unit.



References:

1. M. Morris Mano, Computer System Architecture, Pearson